CPE 344 | PROJECT DELIVERABLE 1

Engineering Baseline and Working Sprint 1

QuickOrder System

Project information	Value
Team member 1	Patiharn Liangkobkit — Student ID 66070503489 — @Richyboy170
Team member 2	Tanakrit Sunthonwetchapong — Student ID 67070503464 — @Tanakrit-triton
Selected context	QuickOrder food-ordering system
Repository	https://github.com/Richyboy170/Software-Engineering-Project
Evidence cut-off	14 November 2026 (Asia/Bangkok)
Release reference	No release tag found. Current main and sprint-1-staging commit: 1e0c693ad0161c4464a5ba6bc7e68720cfa6f584.

Evidence status: verified repository and GitHub facts are separated from historical records; outstanding requirements are identified.

Contents and navigation

Page ranges are generated from this PDF. IDs identify the main requirements and contracts covered in each part.

Part	PDF page range	Contents / IDs
A — SRS and product scope	2–3	Actors, scope, FR/BR/NFR, assumptions and decisions
B — Feature inventory and traceability	4–6	F-01–F-11; FR/BR/NFR mapping
C — System-Level SDS	7–10	Architecture, data, UI/API, access, configuration and delivery
D — Three-sprint plan	11	Sprint goals, features, dependencies and outcomes
E — Sprint 1 engineering contracts	12–15	Feature contracts, AC, STS and Product Definition of Done
F — Working Sprint 1 and verification	16–19	Tests, GitHub workflow, contributions, limitations and gaps

PART A — SRS and product scope

Complete Software Requirements Specification from the project repository.

Software Requirements Specification (SRS) - QuickOrder System

Course: CPE 344 Introduction to Software Engineering in the Age of AI Agents

Project: Deliverable 1 — Engineering Baseline and Working Sprint 1

Team Members:

- Patiharn Liangkobkit (Student ID: 66070503489, GitHub: @Richyboy170 (https://github.com/Richyboy170))
- Tanakrit Sunthonwetchapong (Student ID: 67070503464, GitHub: @Tanakrit-triton (https://github.com/Tanakrit-triton))

Repository: Richyboy170/Software-Engineering-Project (https://github.com/Richyboy170/Software-Engineering-Project)

Theme/Domain: QuickOrder (On-Demand Food & Beverage Ordering Platform)

1. Context and Stakeholder Problem

1.1 Problem Statement

Campus eateries and quick-service dining establishments suffer from severe peak-hour order processing bottlenecks:

- Walk-in customers endure prolonged queuing times just to browse the menu, verify stock availability, place orders, and pay.
- Counter and kitchen staff manage orders through handwritten notes or uncoordinated verbal calls, resulting in lost orders, double-sold items, and lack of preparation tracking.
- Out-of-stock items are frequently sold before the kitchen realizes inventory is depleted, leading to customer frustration and awkward manual cancellation/refund handling.

1.2 Proposed Solution

QuickOrder is an integrated full-stack web ordering system and kitchen dispatch monitor that digitizes both the customer ordering journey and kitchen operations:

- Customers authenticate securely, browse real-time item availability, place orders with atomic inventory deduction, and monitor preparation status live.
- Kitchen and counter staff monitor orders in a real-time queue, transition preparation states (PENDING -> PREPARING -> READY -> COMPLETED), and maintain accurate inventory counts.

2. Actors and Responsibilities

Actor	Description & Core Responsibilities
Customer (CUSTOMER)	Authenticated consumer who browses active menu items, places orders against live inventory, views personal order history, and cancels pending orders within the allowable window.
Staff (STAFF)	Store/kitchen staff member who inspects the global order dispatch queue, updates order preparation statuses, views live stock levels, and fulfills customer orders.
System (Automated / Background)	Enforces transactional consistency, validates stock deductions, calculates authoritative order totals, validates session integrity, and provides health probes.

3. Product Scope and Explicit Exclusions

3.1 In Scope (Baseline across 3 Sprints)

- Role-based authentication (Customer, Staff) with bcrypt password hashing and secure session tokens.
- Menu catalog with item descriptions, prices, categories, and real-time inventory counts.
- Customer ordering vertical slice: order construction, server-side price computation, atomic inventory deduction, and order placement in PENDING state.
- Kitchen dispatch order queue with status transitions (PENDING, PREPARING, READY, COMPLETED, CANCELLED).
- Automated tests across unit, integration, and security layers enforcing domain business rules.
- Production Docker containerization, reproducible migrations/seed, and GCP CI/CD deployment pipeline.

3.2 Explicit Exclusions (Out of Scope)

- Real external banking/credit card payment gateway processing (payment is simulated via mock confirmation).
- External SMS notifications or physical thermal receipt printers.
- Third-party multi-restaurant delivery driver dispatch logistics.
- Self-service email verification links and SMS password resets (synthetic seeded test accounts used for local and UAT evaluation).

4. Functional Requirements (FR)

ID	Title	Observable Behavior & Description	Target Sprint
FR-01	User Authentication	Users authenticate with email and password. System validates credentials and returns session	Sprint 1

		identifying user role (CUSTOMER or STAFF).	
FR-02	User Profile & Identity	Authenticated users retrieve their identity, profile, and role via /api/auth/me. Session terminates cleanly upon logout.	Sprint 1
FR-03	Menu Catalog Browsing	Users browse active menu items including name, description, price, category, and real-time stock count.	Sprint 1
FR-04	Order Placement	Customers submit an order with one or more menu items and quantities. Server validates inventory and creates an order in PENDING status.	Sprint 1
FR-05	Customer Order History	Customers view and inspect the history and line-item details of their own submitted orders.	Sprint 1
FR-06	Staff Order Queue & Status Updates	Staff view all orders across all customers and advance preparation status (PENDING -> PREPARING -> READY -> COMPLETED).	Sprint 1
FR-07	Customer Order Cancellation	Customers cancel their own order only while its status is PENDING. Stock is restored upon cancellation.	Sprint 1
FR-08	Menu & Stock Inventory Management	Staff create new menu items, update prices, and restock inventory.	Sprint 2
FR-09	Digital Order Receipt & Pickup Code	System generates printable order summary with customer details and a unique 4-character pickup reference code.	Sprint 2
FR-10	Sales Analytics & Reporting	Staff review aggregated daily sales revenue, order volumes, and top-performing items.	Sprint 3

5. Business Rules (BR)

ID	Title	Rule Definition & Boundary Behavior
BR-01	Minimum Order Content	An order must contain at least 1 line item, and each line item quantity must be an integer greater than zero ($q \ge 1$). Submissions with empty items or non-positive quantities are rejected with HTTP 400 Bad Request.
BR-02	Atomic Stock Deduction	An order cannot be placed if requested quantity exceeds current available stock for any item. If stock is sufficient, the system atomically reserves and deducts stock upon order creation. Zero or negative remaining stock prevents further orders.
BR-03	Server-Authoritative Pricing	Total order price and line item unit prices must be computed exclusively from server-side database records. Any prices submitted in the client payload are ignored to prevent price tampering.
BR-04	Data Isolation & Ownership	Customers may only read and modify their own orders. Attempting to query or cancel another user's order must return HTTP 403 Forbidden without disclosing order contents.
BR-05	Order Cancellation Window	A customer can only cancel an order whose current status is PENDING. Once an order is transitioned to PREPARING, READY, or COMPLETED, customer cancellation is rejected with HTTP 400.
BR-06	Restricted State Progression	Only users with the STAFF role can advance order states (PENDING -> PREPARING -> READY -> COMPLETED). Customers attempting status promotion receive HTTP 403 Forbidden.

6. Non-Functional Requirements (NFR)

ID	Category	Measurable Target	Practical Verification Method	Target Sprint
NFR-01	Security / Access Control	Unauthenticated requests to protected endpoints	Automated Supertest API security suite.	Sprint 1

		(/api/orders/, /api/auth/me) are rejected with HTTP 401 Unauthorized without leaking private record data.		
NFR-02	Security / Role Restriction	Customer role users attempting Staff-only endpoints (/api/orders/:id/status, menu item mutations) are rejected with HTTP 403 Forbidden.	Automated Supertest API role boundary tests.	Sprint 1
NFR-03	Performance / Response Time	API endpoints for menu listing and order submission must respond within $\le 200\text{ ms}$ under local automated test suites.	Supertest API benchmark test.	Sprint 1
NFR-04	Data Integrity & Concurrency	Simultaneous order requests competing for the last unit of stock must not oversell inventory (zero negative stock tolerance).	Concurrency race condition test with transactional lock.	Sprint 2
NFR-05	Cloud Readiness & Portability	Application runs from a clean container build; configuration injected exclusively via environment variables without hardcoded secrets.	Docker fresh build verification in GitHub Actions CI (test-and-docker).	Sprint 1 / Lab 5
NFR-06	Disaster Recovery & Persistence	All committed order and inventory records survive container restarts and Cloud Run redeployments.	Post-restart database retrieval test (Lab 5 verification C-05).	Lab 5

7. Assumptions, Decisions, and Resolutions

1. Architecture & Stack Decision:

Resolution: Node.js (TypeScript) + Express for the REST API, Prisma ORM with PostgreSQL for robust transactional support, and React + Vite for responsive UI. This aligns 100% with the provided toktickit-lab5-starter containerization and GCP CI/CD templates.

2. Authentication Strategy:

Resolution: Session-based token/cookie with bcrypt-hashed passwords in the database. Allows real server-side authentication, role inspection, and session termination without requiring third-party OAuth in Sprint 1.

3. Inventory Race Prevention:

Resolution: Prisma interactive transactions (prisma.$transaction) are used during order creation to inspect stock, check constraints, deduct inventory, and create order records atomically.

PART B — Feature inventory and traceability

Feature definitions and the requirements-to-feature-to-sprint map are reproduced below.

Feature inventory

Feature Inventory - QuickOrder System

Course: CPE 344 Deliverable 1

Team Members:

- Patiharn Liangkobkit (66070503489, GitHub: @Richyboy170 (https://github.com/Richyboy170))
- Tanakrit Sunthonwetchapong (67070503464, GitHub: @Tanakrit-triton (https://github.com/Tanakrit-triton))

Domain: QuickOrder System

1. Feature Catalog

Feature ID	Feature Name	Short Description	Related Requirements	Dependencies	Planned Sprint
F-01	User Authentication & Identity	Login with email and password, secure password verification via bcrypt, role retrieval (/api/auth/me), and logout session	FR-01, FR-02, NFR-01, NFR-02	None	Sprint 1

		destruction.			
F-02	Menu Item Catalog & Live Stock	Display available food and beverage items with item names, prices, descriptions, category, and real-time stock levels.	FR-03, NFR-03	Database setup	Sprint 1
F-03	Order Creation & Inventory Validation	Customer selects items & quantities; backend validates non-empty payload, verifies and atomically deducts stock, calculates server-authoritative totals, and stores order in PENDING state.	FR-04, BR-01, BR-02, BR-03, NFR-01, NFR-04	F-01, F-02	Sprint 1
F-04	Order History & Customer Isolation	Customers can list and inspect their past orders. Non-owners cannot view or access another customer's order records.	FR-05, BR-04, NFR-01	F-01, F-03	Sprint 1
F-05	Customer Order Cancellation	Allows customers to cancel a placed order if and only if it remains in PENDING status. Restores inventory stock upon cancellation.	FR-07, BR-05	F-03, F-04	Sprint 1
F-06	Staff Kitchen Order Queue & Status Lifecycle	Staff dashboard listing all incoming orders; provides controls to transition order status (PENDING -> PREPARING -> READY -> COMPLETED).	FR-06, BR-06, NFR-02	F-01, F-03	Sprint 1
F-07	Menu & Stock Inventory Management	Staff interface to create new items, update descriptions, adjust prices, and restock quantities.	FR-08	F-01, F-02	Sprint 2
F-08	Digital Order Receipt & Order Code	Printable receipt generation with order summary, timestamp, customer information, and unique pickup confirmation code.	FR-09	F-03	Sprint 2
F-09	Concurrency Protection & High-Load Inventory Locking	Explicit optimistic/pessimistic lock mechanisms to prevent inventory overselling during high concurrent order bursts.	NFR-04	F-03	Sprint 2
F-10	Sales Reporting & Product Analytics	Aggregated daily and weekly sales reports, revenue trends, and top-selling menu items for store management.	FR-10	F-03, F-06	Sprint 3
F-11	System Hardening & Observability	Structured logging, readiness/liveness health probes, rate limiting, and GCP deployment pipeline monitoring.	NFR-05, NFR-06	F-01..F-10	Sprint 3

2. Sprint 1 Scope Justification

Sprint 1 establishes the end-to-end engineering vertical slice:

4. Full stack integration: UI (React), API (Express), Database (PostgreSQL + Prisma).
5. Two distinct user roles: CUSTOMER (orders items, views personal history, cancels pending) and STAFF (monitors queue, advances kitchen preparation status).

6. Core domain transaction & business rules: Atomic stock validation, server-side price calculation, customer data isolation, and status progression guards.

Requirements traceability

Requirements-to-Feature-to-Sprint Traceability Matrix

Course: CPE 344 Deliverable 1

Team Members:

- Patiharn Liangkobkit (66070503489, GitHub: @Richyboy170 (https://github.com/Richyboy170))
- Tanakrit Sunthonwetchapong (67070503464, GitHub: @Tanakrit-triton (https://github.com/Tanakrit-triton))

Domain: QuickOrder System

1. Traceability Matrix

Requirement ID	Type	Requirement Summary	Feature ID & Responsibility	Planned Sprint	Verification Method / Planned Test ID
FR-01	FR	User Authentication (email + password)	F-01 User Auth & Identity	Sprint 1	AC-AUTH-01 / TEST-AUTH-01 (Supertest API)
FR-02	FR	User Session & Logout	F-01 User Auth & Identity	Sprint 1	AC-AUTH-02 / TEST-AUTH-02 (Supertest API)
FR-03	FR	Menu Catalog Browsing with Stock	F-02 Menu Item Catalog	Sprint 1	AC-MENU-01 / TEST-MENU-01 (Supertest API + UI)
FR-04	FR	Order Placement & Total Computation	F-03 Order Creation	Sprint 1	AC-ORD-01 / TEST-ORD-01 (Integration Test)
FR-05	FR	Customer Order History	F-04 Order History	Sprint 1	AC-ORD-02 / TEST-ORD-02 (Supertest API + UI)
FR-06	FR	Staff Order Queue & Status Update	F-06 Staff Kitchen Queue	Sprint 1	AC-STAFF-01 / TEST-STAFF-01 (Supertest API + UI)
FR-07	FR	Customer Order Cancellation (Pending)	F-05 Order Cancellation	Sprint 1	AC-ORD-03 / TEST-ORD-03 (Integration Test)
FR-08	FR	Menu Item & Stock Management	F-07 Menu Inventory Management	Sprint 2	Sprint 2 Contract & Test Suite
FR-09	FR	Order Receipt & Pickup Code	F-08 Digital Order Receipt	Sprint 2	Sprint 2 Contract & Test Suite
FR-10	FR	Sales Reporting & Analytics	F-10 Sales Analytics	Sprint 3	Sprint 3 Contract & Test Suite
BR-01	BR	Order Must Have ≥ 1 Positive Item	F-03 Order Creation	Sprint 1	AC-ORD-04 / TEST-ORD-04 (Unit + API Test)
BR-02	BR	Insufficient Stock Rejection	F-03 Order Creation	Sprint 1	AC-ORD-05 / TEST-ORD-05 (Integration Test)
BR-03	BR	Server-Authoritative Price Calculation	F-03 Order Creation	Sprint 1	AC-ORD-06 / TEST-ORD-06 (API Price Tamper Test)
BR-04	BR	Customer Data Isolation (No cross-access)	F-04 Order History	Sprint 1	AC-SEC-01 / TEST-SEC-01 (Forbidden Access Test)
BR-05	BR	Cancel Allowed Only When PENDING	F-05 Order Cancellation	Sprint 1	AC-ORD-07 / TEST-ORD-07 (Status Boundary Test)
BR-06	BR	Only Staff May Promote Status	F-06 Staff Kitchen Queue	Sprint 1	AC-SEC-02 / TEST-SEC-02 (Role Permission Test)
NFR-01	NFR	Protected Endpoints Reject Unauth (401)	System Responsibility: Auth Middleware	Sprint 1	AC-SEC-03 / TEST-SEC-03 (Supertest API Security)
NFR-02	NFR	Customer Forbidden on Staff Endpoints (403)	System Responsibility: Role Middleware	Sprint 1	AC-SEC-04 / TEST-SEC-04 (Supertest API Security)
NFR-03	NFR	Response Time $\le 200\text{ ms}$	System Responsibility: API Layer	Sprint 1	AC-PERF-01 / TEST-PERF-01 (API Benchmark)
NFR-04	NFR	Zero Negative Stock Tolerance	F-09 Concurrency Locking	Sprint 2	Concurrency Test Suite
NFR-05	NFR	Container Packaging & Zero Hardcoded Secrets	System Responsibility: DevOps / Docker	Sprint 1 / Lab 5	GitHub Actions CI test-and-docker
NFR-06	NFR	Data Persistence	System	Lab 5	Lab 5 Verification

	across Container Restart	Responsibility: Cloud SQL / Storage	Check C-05

2. Coverage Gaps and Scope Baseline Analysis

- Sprint 1 Coverage:

Covers 100% of foundational requirements: Authentication (FR-01, FR-02), Menu Catalog (FR-03), Order Submission & Retrieval (FR-04, FR-05), Staff Queue (FR-06), Order Cancellation (FR-07), all core Business Rules (BR-01 to BR-06), and baseline security/performance NFRs (NFR-01, NFR-02, NFR-03, NFR-05).

- Sprint 2 Coverage:

Covers expansion features: Staff-side menu editing/restocking (FR-08), printable digital receipt (FR-09), and high-concurrency race condition testing (NFR-04).

- Sprint 3 Coverage:

Covers product hardening: Analytics dashboard (FR-10), structured audit logging, rate limiting, and final release tuning (F-11).

- Explicit Exclusions / Approved Deferrals:

Third-party payment gateways and physical thermal printing hardware are excluded from product scope by architectural design decision. Simulated payment confirmations are utilized.

3. Sprint 2 Persistence Foundation Traceability

The persistence foundation in Issue #4 (https://github.com/Richyboy170/Software-Engineering-Project/issues/4) makes the existing F-01 through F-06 journey durable before Sprint 2 expansion work. The acceptance IDs below are defined in the Sprint 2 persistence contract (./sprints/sprint-2/persistence-spec.md).

Acceptance ID	Source requirement / rule	Persistence behavior	Verification test
AC-PERSIST-01	F-01; test architecture constraint	Two apps can share the in-memory adapter without changing fixture-based tests.	TEST-PERSIST-01
AC-PERSIST-02	NFR-06; Lab 5 C-05	Migrations and idempotent seed provision the PostgreSQL demo fixtures.	TEST-PERSIST-02
AC-PERSIST-03	FR-04; BR-03	Order lines preserve the name and price captured at order time.	TEST-PERSIST-03
AC-PERSIST-04	FR-01; FR-02; NFR-01	Login, cross-instance session lookup, expiry, and logout revocation use PostgreSQL.	TEST-PERSIST-04
AC-PERSIST-05	FR-03	Menu reads return database-backed stock availability and numeric prices.	TEST-PERSIST-05
AC-PERSIST-06	FR-04; BR-03	Successful orders use database prices, persist across instances, and reserve stock once.	TEST-PERSIST-06
AC-PERSIST-07	BR-01; BR-02; NFR-04	Invalid, insufficient, duplicate-line, and final-unit race cases cannot partially write or oversell.	TEST-PERSIST-07
AC-PERSIST-08	FR-05; FR-06; FR-07; BR-04; BR-05; BR-06	Customer ownership and pending-only cancellation, one-time stock restoration, and staff-only forward status changes remain enforced.	TEST-PERSIST-08
AC-PERSIST-09	NFR-06; Lab 5 readiness check	Readiness reflects database connectivity while liveness remains independent.	TEST-PERSIST-09

PART C — System-Level SDS

The complete system-level design document is reproduced below.

System-Level Software Design Specification (SDS) - QuickOrder

Course: CPE 344 Deliverable 1

Team Members:

- Patiharn Liangkobkit (66070503489, GitHub: @Richyboy170 (https://github.com/Richyboy170))
- Tanakrit Sunthonwetchapong (67070503464, GitHub: @Tanakrit-triton (https://github.com/Tanakrit-triton))

Theme/Domain: QuickOrder Ordering Platform

Target Checkpoint: Deliverable 1 (8 November 2026) / Lab 5 CI/CD Ready

1. System Architecture and Component Responsibilities

1.1 Architecture Overview

QuickOrder employs a clean client-server architecture organized into three decoupled tiers:

7. Presentation Layer (Client): A single-page application built using React 18 + Vite + Tailwind CSS. It manages UI state, form validation, cart staging, dynamic status polling, and session tokens.
8. Application / API Layer (Server): Built on Node.js (TypeScript) + Express. It enforces all authentication, role permissions, request validation, transaction boundaries, and business rules (BR-01 through BR-06).
9. Persistence Layer (Database): A relational PostgreSQL 16 database managed through Prisma ORM. It enforces relational integrity, unique constraints, foreign keys, and atomic operations.

```
flowchart TD
    subgraph Client ["Client Layer (React / Vite)"]
        UI["Web Browser / Responsive UI"]
        CartState["Cart & Session Storage"]
    end

    subgraph Server ["Application Layer (Express / Node.js)"]
        AuthMiddleware["Auth & Role Guard Middleware"]
        Router["REST API Router (/api/*)"]
        OrderService["Order & Business Rule Engine"]
        HealthRouter["Health & Readiness Probes (/api/health, /api/ready)"]
    end

    subgraph Database ["Persistence Layer (PostgreSQL / Prisma)"]
        Prisma["Prisma ORM Client"]
        Postgres[(PostgreSQL 16 Database)]
    end

    UI -->|HTTP / JSON Requests| AuthMiddleware
    AuthMiddleware --> Router
    Router --> OrderService
    OrderService --> Prisma
    Prisma -->|SQL via Unix Socket / TCP| Postgres
    Router --> HealthRouter
    HealthRouter --> Prisma
```

1.2 Enforcement Boundaries

- No Client Trust: The frontend validates input for user experience only. The backend API independently validates all payloads (quantity ≥ 1, valid IDs, inventory sufficiency) and authoritatively computes pricing.
- Transactional Consistency: Order placement executes within a single prisma.$transaction(...) block. Stock availability is verified and decremented in lockstep with Order and OrderItem record insertion.
- Fail-Safe Rollback: If any item exceeds stock, the transaction immediately aborts, no database changes are committed, and an HTTP 400 error is returned.

2. Domain Data Model and Schema Specification

2.1 Entity Relationship Diagram

```
erDiagram
    USER ||--o{ SESSION : has
    USER ||--o{ ORDER : places
    ORDER ||--|{ ORDER_ITEM : contains
    MENU_ITEM ||--o{ ORDER_ITEM : referenced_by

    USER {
        string id PK
        string email UK
        string passwordHash
        string fullName
        string role "CUSTOMER | STAFF"
        datetime createdAt
        datetime updatedAt
    }

    SESSION {
        string token PK
        string userId FK
        datetime expiresAt
        datetime createdAt
    }

    MENU_ITEM {
        string id PK
        string name
        string description
        string category
        decimal price
```

```
    int stock
    boolean isAvailable
    datetime createdAt
    datetime updatedAt
}

ORDER {
    string id PK
    string customerId FK
    string status "PENDING | PREPARING | READY | COMPLETED | CANCELLED"
    decimal totalPrice
    datetime createdAt
    datetime updatedAt
}

ORDER_ITEM {
    string id PK
    string orderId FK
    string menuItemId FK
    int quantity
    decimal unitPrice
    decimal subtotal
}
```

2.2 Data Dictionary and Key Constraints

10. User Table:

- id: CUID/UUID string primary key.
- email: Unique string (@ format required).
- passwordHash: Bcrypt salt+hash (minimum 10 rounds). Passwords never stored plaintext.
- role: Enum CUSTOMER or STAFF.

11. Session Table:

- token: Unique cryptographically generated hex token.
- userId: Foreign key to User.id on delete CASCADE.
- expiresAt: DateTime (default 24 hours expiry).

12. MenuItem Table:

- id: CUID/UUID primary key.
- name: Non-empty string.
- price: Decimal $(10, 2)$ strictly > 0.
- stock: Integer ≥ 0. Constraint: negative values rejected by DB check.
- isAvailable: Boolean flag.

13. Order Table:

- id: CUID primary key.
- customerId: Foreign key to User.id.
- status: Enum (PENDING, PREPARING, READY, COMPLETED, CANCELLED).
- totalPrice: Decimal $(10, 2)$, sum of all line item subtotals calculated server-side.

14. OrderItem Table:

- orderId: Foreign key to Order.id on delete CASCADE.
- menuItemId: Foreign key to MenuItem.id.
- quantity: Integer ≥ 1.
- unitPrice: Snapshot of MenuItem.price at moment of purchase.
- subtotal: quantity $\times$ unitPrice.

3. UI, API, and Access Conventions

3.1 Main Screens & Shared UI Behavior

- Sign In / Registration Screen: Clean form with email and password inputs, role badge display for seeded accounts, immediate client validation, and clear error banner upon 401.
- Menu Catalog Screen: Grid view displaying items, categories, price badges, live stock counter, and "Add to Cart" actions. Out-of-stock items have disabled add buttons.
- Cart & Order Review Modal/Drawer: Summarizes selected quantities, live total, and a "Place Order" button with loading spinners to prevent double-click submissions.
- Customer My Orders Screen: Lists the authenticated customer's orders with current status badges, line-item summaries, timestamps, and a conditional "Cancel Order" button (enabled only when status is PENDING).
- Staff Kitchen Queue Dashboard: Live operational board displaying all orders grouped by status, with single-click status advance buttons (Start Preparing, Mark Ready, Complete).
- Shared States: Every asynchronous query exhibits 3 explicit UI states:
- Loading State: Accessible skeleton placeholders or spinner.
- Empty State: Meaningful messaging (e.g., "You have no active orders yet").
- Error State: Human-readable alert with retry action.

3.2 REST API Endpoint Conventions

All API endpoints follow RESTful standards, utilize JSON payloads, and return predictable envelopes:

- Success Response:

```
{
  "success": true,
  "data": { ... }
}
```

- Error Response:

```
{
  "success": false,
  "error": {
    "code": "INSUFFICIENT_STOCK",
    "message": "Requested quantity exceeds available stock."
  }
}
```

Core Endpoint Routes

Method	Path	Auth Required	Allowed Role	Purpose
GET	/api/health	No	Any	System liveness probe (status: ok)
GET	/api/ready	No	Any	DB readiness probe (SELECT 1)
POST	/api/auth/login	No	Any	User login, returns session token
GET	/api/auth/me	Yes	Any	Returns authenticated user profile
POST	/api/auth/logout	Yes	Any	Invalidates session token
GET	/api/menu	No	Any	Retrieves active menu items & stock
POST	/api/orders	Yes	CUSTOMER	Submits new order (validates stock & calculates price)
GET	/api/orders/my	Yes	CUSTOMER	Lists current customer's orders
GET	/api/orders	Yes	STAFF	Lists all customer orders for queue
GET	/api/orders/:id	Yes	Owner / STAFF	Retrieves single order details
POST	/api/orders/:id/cancel	Yes	Owner	Cancels order (only if PENDING)
PATCH	/api/orders/:id/status	Yes	STAFF	Updates order state progression

4. Role and Permission Matrix

Operation / Resource	Unauthenticated	Customer Role	Staff Role
View API Health (/api/health, /api/ready)	☑ Yes	☑ Yes	☑ Yes
View Menu Items & Stock (GET /api/menu)	☑ Yes	☑ Yes	☑ Yes
Sign In / Sign Out	☑ Sign In	☑ Sign Out	☑ Sign Out
Place Order (POST /api/orders)	✗ 401	☑ Yes	✗ 403 (Customer only)
View Own Orders (GET /api/orders/my)	✗ 401	☑ Yes	✗ 403
View All Orders (GET /api/orders)	✗ 401	✗ 403 Forbidden	☑ Yes
View Order Details (GET /api/orders/:id)	✗ 401	☑ Own Only (403 if other)	☑ Any Order
Cancel Order (POST /api/orders/:id/cancel)	✗ 401	☑ Own & PENDING only	✗ 403
Advance Status (PATCH /api/orders/:id/status)	✗ 401	✗ 403 Forbidden	☑ Yes

5. Security and Credential Handling

15. Password Security: Passwords hashed with bcryptjs using a minimum work factor of 10. Passwords are never returned in responses or logged.
16. Session Authentication:

- Client sends session token via Authorization: Bearer <token> or x-session-token header.
- Express middleware queries the Session table, checks expiresAt > NOW(), and attaches req.user to the request object.
- Logout removes the session token from the database, immediately invalidating any subsequent requests.

17. Safe Seed Credentials:

- Pre-seeded accounts are provided for local development and TA evaluation (customer@quickorder.local, staff@quickorder.local). Passwords default to known non-production credentials locally and are driven by environment variables in cloud environments.

6. Configuration and Cloud Delivery Design

6.1 Environment Variables

All configuration is loaded via environment variables; no secrets are committed to Git:

Variable	Description	Example / Local Default
PORT	Listening HTTP port for the container	3000
NODE_ENV	Application mode (development or production)	development
DATABASE_URL	PostgreSQL connection string	postgresql://quickorder:devpassword@localhost:5432/quickorder
CORS_ORIGIN	Allowed client origins	http://localhost:5173
SESSION_SECRET	Secret used for session generation	quickorder-super-secret-key-2026

6.2 Cloud Packaging and Target GCP Architecture (Lab 5 Ready)

- Containerization: Multi-stage Docker build packaging both static React client build and Express backend into a single lean container.
- Hosting: Google Cloud Run (serverless container hosting with 0-to-3 instance scaling).
- Database: Google Cloud SQL (PostgreSQL 16) with storage auto-increase disabled.
- Secrets: Google Secret Manager stores DATABASE_URL and admin credentials.
- CI/CD: GitHub Actions with Workload Identity Federation (WIF) ensuring keyless zero-credential deployment.

PART D — Three-sprint plan

The complete three-sprint plan is reproduced below.

Three-Sprint Engineering Plan - QuickOrder System

Course: CPE 344 Deliverable 1

Team Members:

- Patiharn Liangkobkit (66070503489, GitHub: @Richyboy170 (https://github.com/Richyboy170))
- Tanakrit Sunthonwetchapong (67070503464, GitHub: @Tanakrit-triton (https://github.com/Tanakrit-triton))

Domain: QuickOrder System

1. Executive Summary & Delivery Schedule

Sprint	Objective & Milestone	Target Checkpoint	Feature IDs	Primary Completion Outcome
Sprint 1	Foundational Vertical Slice & Core Rules: Authentication, Menu Catalog, Order Placement, Kitchen Queue, Status Progression, and Strict Business Rules.	Deliverable 1 (8 Nov 2026)	F-01, F-02, F-03, F-04, F-05, F-06	Working end-to-end user journey across UI, API, and DB with automated tests, reproducible seed data, and reviewed Git integration.
Sprint 2	Core Expansion & Cloud Pipeline: Staff Menu/Stock Management, Digital Receipts/Pickup Codes, High-Concurrency Locking, and GCP Cloud Deployment with CI/CD quality gate.	Deliverable 2 / Lab 5 (15 Nov - 29 Nov 2026)	F-07, F-08, F-09, NFR-04, NFR-05, NFR-06	Live GCP deployment on Cloud Run + Cloud SQL; PR merge gate with automated test checks; repeatable release pipeline.
Sprint 3	Product Hardening & Operations: Sales Analytics, Audit Logging, Rate Limiting, User Acceptance Testing (UAT), Defect Remediation, and Final Presentation.	Progress (29 Nov), Final Complete (13 Dec 2026)	F-10, F-11	Fully hardened production system, passing all regression and UAT scenarios, comprehensive documentation, and final demo video.

2. Detailed Sprint Work Breakdown

```
flowchart LR
    subgraph Sprint1 ["Sprint 1: Baseline Vertical Slice"]
        S1A["F-01 Auth & Sessions"] --> S1B["F-02 Menu & Stock"]
        S1B --> S1C["F-03 Order Placement & BRs"]
        S1C --> S1D["F-04 Customer Order History"]
        S1C --> S1E["F-05 Order Cancellation"]
        S1C --> S1F["F-06 Staff Kitchen Queue"]
```

```
    end

    subgraph Sprint2 ["Sprint 2: Core Expansion & Cloud CI/CD"]
        S2P["Persistence foundation (Issue #4)"]
        S2A["F-07 Menu Management"]
        S2B["F-08 Receipts & Pickup Code"]
        S2C["F-09 Concurrency Locking"]
        S2D["Lab 5 GCP Pipeline & Docker"]
        S2P --> S2A
        S2P --> S2B
        S2P --> S2C
        S2P --> S2D
    end

    subgraph Sprint3 ["Sprint 3: Hardening & Analytics"]
        S3A["F-10 Sales Analytics"]
        S3B["F-11 Hardening & Rate Limiting"]
        S3C["UAT & Defect Triage"]
        S3D["Final Delivery & Presentation"]
    end

    Sprint1 --> Sprint2
    Sprint2 --> Sprint3
```

2.1 Sprint 1: Baseline Vertical Slice

- Goal: Deliver an authenticated, transactional vertical slice connecting the React frontend, Express API backend, and PostgreSQL database.
- Included Features:
- F-01: Customer and Staff authentication, password hashing, session tokens, current-user profile (/api/auth/me), and logout.
- F-02: Real-time menu browsing with live inventory availability.
- F-03: Customer cart checkout, atomic stock verification and deduction, server-side price computation, order placement in PENDING state.
- F-04: Customer personal order history with strict data isolation.
- F-05: Customer order cancellation while in PENDING state with stock restoration.
- F-06: Staff order queue with state progression (PENDING -> PREPARING -> READY -> COMPLETED).
- Dependencies: Node.js runtime, PostgreSQL, Prisma schema migrations, seed dataset.
- Completion Criteria:
- Complete customer flow: Login -> Browse Menu -> Place Order -> View Order -> Cancel Order.
- Complete staff flow: Login -> View Kitchen Queue -> Advance Preparation Status.
- Passing automated test suite for unit, integration, and security role permissions.
- Local setup verified via Docker Compose.

2.2 Sprint 2: Core Expansion & Cloud Delivery

- Goal: Broaden administrative capabilities and deploy the system to Google Cloud Platform via a verified CI/CD pipeline.
- Persistence prerequisite: Before adding Sprint 2 features or proceeding with Cloud SQL delivery, make the existing F-01 through F-06 production journey durable through Prisma and PostgreSQL. Track this foundation in Issue #4 (https://github.com/Richyboy170/Software-Engineering-Project/issues/4) and the Sprint 2 persistence contract (./sprint-2/persistence-spec.md). It preserves the existing in-memory unit-test fixtures and follows Lab 5's issue-linked feature/prisma-persistence → sprint-1-staging review flow.
- Included Features:
- F-07: Staff CRUD controls for menu items (create, update description/price, restock quantities).
- F-08: Printable digital receipts and unique pickup order reference codes.
- F-09: Database concurrency locking to guarantee zero overselling under simultaneous requests.
- Cloud Delivery: Cloud Run packaging, Cloud SQL persistence, Secret Manager integration, and GitHub Actions PR check rulesets (test-and-docker).
- Dependencies: Completion of the Sprint 1 release baseline on main; completion and peer review of the persistence prerequisite on sprint-1-staging before Cloud SQL delivery. The persistence foundation does not include F-07, F-08, the broad F-09 concurrency program, or GCP deployment changes.

2.3 Sprint 3: Hardening, Analytics & Final Delivery

- Goal: Fortify system security and resilience, implement operational reporting, execute peer UAT, and finalize product delivery.
- Included Features:
- F-10: Administrative sales reporting and revenue analytics.
- F-11: Structured logging, request rate limiting, input sanitization, and automated smoke test verifications.
- Comprehensive UAT execution with partner team and defect triage log.
- Dependencies: Sprint 2 cloud deployment.

PART E — Sprint 1 engineering contracts

Feature engineering contracts and the Software Test Specification are reproduced below. Completion status is reconciled in Part F against current GitHub and branch evidence.

Feature contracts and acceptance criteria

Sprint 1 Feature Engineering Contracts & Specifications

Course: CPE 344 Deliverable 1

Team Members:

- Patiharn Liangkobkit (66070503489, GitHub: @Richyboy170 (https://github.com/Richyboy170))
- Tanakrit Sunthonwetchapong (67070503464, GitHub: @Tanakrit-triton (https://github.com/Tanakrit-triton))

Target Milestone: Sprint 1 Release on main

Domain: QuickOrder System

1. Feature Specifications

1.1 Feature F-01: User Authentication & Session Management

- Linked Requirements: FR-01, FR-02, NFR-01, NFR-02
- Description: Provides secure user sign-in using email and password, session token issuance, current authenticated user profile inspection, and session termination upon logout.
- Roles & Permissions: Open to public for login; all authenticated roles can query /api/auth/me and logout.
- API Specification:
- POST /api/auth/login
- Request: { "email": "customer@quickorder.local", "password": "Password123!" }
- Response (200): { "success": true, "data": { "token": "hex-token", "user": { "id": "...", "email": "...", "fullName": "...", "role": "CUSTOMER" } } }
- Error (401): { "success": false, "error": { "code": "INVALID_CREDENTIALS", "message": "Invalid email or password." } }
- GET /api/auth/me
- Header: Authorization: Bearer <token>
- Response (200): { "success": true, "data": { "id": "...", "email": "...", "fullName": "...", "role": "CUSTOMER" } }
- Error (401): { "success": false, "error": { "code": "UNAUTHORIZED", "message": "Authentication required." } }
- POST /api/auth/logout
- Header: Authorization: Bearer <token>
- Response (200): { "success": true, "data": { "message": "Logged out successfully." } }

Acceptance Criteria

- AC-AUTH-01: Given valid credentials for an existing user, posting to /api/auth/login must return HTTP 200, a non-empty session token, and the user profile containing the assigned role.
- AC-AUTH-02: Given incorrect password or non-existent email, posting to /api/auth/login must return HTTP 401 without creating a session.
- AC-AUTH-03: An authenticated user providing a valid token to /api/auth/me receives HTTP 200 with their own user details.
- AC-AUTH-04: Calling /api/auth/logout revokes the token in the database; subsequent requests using that token must return HTTP 401.

1.2 Feature F-02: Menu Catalog & Real-Time Stock Inspection

- Linked Requirements: FR-03, NFR-03
- Description: Allows any user (guest or authenticated) to view active food and beverage offerings, current prices, descriptions, and remaining inventory stock.
- API Specification:
- GET /api/menu
- Response (200):

```
{
  "success": true,
  "data": [
    {
      "id": "menu_01",
      "name": "Classic Cheeseburger",
      "description": "Grilled beef patty with cheddar and secret sauce",
      "category": "Mains",
      "price": "149.00",
      "stock": 25,
      "isAvailable": true
    }
  ]
}
```

Acceptance Criteria

- AC-MENU-01: Querying GET /api/menu returns HTTP 200 with all active menu items, exact pricing, and non-negative stock counts.
- AC-MENU-02: When stock drops to 0, stock reports 0 and isAvailable evaluates to false in the response.

1.3 Feature F-03: Order Placement & Stock Validation

- Linked Requirements: FR-04, BR-01, BR-02, BR-03, NFR-01, NFR-04
- Description: Authenticated customers place orders with line items. The backend verifies positive quantities, confirms sufficient stock inside an atomic transaction, decrements inventory, computes total price from database prices, and persists the order as PENDING.
- API Specification:

- POST /api/orders
- Header: Authorization: Bearer <token>
- Request:

```
{
  "items": [
    { "menuItemId": "menu_01", "quantity": 2 }
  ]
}
```

- Response (201):

```
{
  "success": true,
  "data": {
    "id": "ord_101",
    "customerId": "usr_01",
    "status": "PENDING",
    "totalPrice": "298.00",
    "items": [
      {
        "id": "item_1",
        "menuItemId": "menu_01",
        "name": "Classic Cheeseburger",
        "quantity": 2,
        "unitPrice": "149.00",
        "subtotal": "298.00"
      }
    ]
  }
}
```

- Error - Empty Items (400): { "success": false, "error": { "code": "INVALID_ITEMS", "message": "Order must contain at least one item with quantity >= 1." } }
- Error - Out of Stock (400): { "success": false, "error": { "code": "INSUFFICIENT_STOCK", "message": "Requested quantity exceeds available stock." } }

Acceptance Criteria

- AC-ORD-01: A valid customer order with sufficient stock returns HTTP 201 with status PENDING, correct server-computed total price, and decrements stock in the database.
- AC-ORD-02: An order submitted with empty items list or non-positive quantities ($q \le 0$) returns HTTP 400 Bad Request (BR-01).
- AC-ORD-03: An order requesting quantity greater than available stock returns HTTP 400 Bad Request, commits no order, and leaves inventory untouched (BR-02).
- AC-ORD-04: Client-tampered price or totalPrice fields in the request body are ignored; the stored total strictly reflects current database item prices (BR-03).

1.4 Feature F-04: Order History & Ownership Isolation

- Linked Requirements: FR-05, BR-04, NFR-01
- Description: Customers view only their own orders. Attempting to query another customer's order directly returns HTTP 403 Forbidden.
- API Specification:
- GET /api/orders/my (Lists orders for authenticated customer)
- GET /api/orders/:id (Returns single order details if requester is owner or Staff)

Acceptance Criteria

- AC-SEC-01: GET /api/orders/my returns HTTP 200 containing only orders where customerId == req.user.id.
- AC-SEC-02: When Customer A attempts GET /api/orders/:id for an order owned by Customer B, the server responds with HTTP 403 Forbidden without disclosing order details (BR-04).

1.5 Feature F-05: Customer Order Cancellation

- Linked Requirements: FR-07, BR-05
- Description: Customers may cancel an order they own if and only if it is in PENDING status. Cancelled orders restore item stock back to inventory.
- API Specification:
- POST /api/orders/:id/cancel
- Header: Authorization: Bearer <token>
- Response (200): { "success": true, "data": { "id": "ord_101", "status": "CANCELLED" } }
- Error - Non-Pending (400): { "success": false, "error": { "code": "INVALID_TRANSITION", "message": "Only PENDING orders can be cancelled." } }

Acceptance Criteria

- AC-ORD-05: A customer cancelling their own PENDING order receives HTTP 200, status becomes CANCELLED, and inventory stock is restored.
- AC-ORD-06: Attempting to cancel an order whose status is PREPARING, READY, or COMPLETED returns HTTP 400 Bad Request (BR-05).

1.6 Feature F-06: Staff Kitchen Queue & State Progression

- Linked Requirements: FR-06, BR-06, NFR-02
- Description: Staff view all customer orders and promote preparation status sequentially (PENDING -> PREPARING -> READY -> COMPLETED).
- API Specification:
- GET /api/orders (Staff only: returns all active orders)
- PATCH /api/orders/:id/status
- Request: { "status": "PREPARING" }
- Response (200): { "success": true, "data": { "id": "...", "status": "PREPARING" } }

Acceptance Criteria

- AC-STAFF-01: A user with STAFF role calling GET /api/orders receives HTTP 200 with all orders across all customers.
- AC-STAFF-02: A user with CUSTOMER role calling GET /api/orders receives HTTP 403 Forbidden (NFR-02).
- AC-STAFF-03: Staff can update status through valid transitions (PENDING -> PREPARING -> READY -> COMPLETED).
- AC-STAFF-04: Customers attempting PATCH /api/orders/:id/status receive HTTP 403 Forbidden (BR-06).

Software Test Specification

Software Test Specification (STS) - Sprint 1

Course: CPE 344 Deliverable 1

Team Members:

- Patiharn Liangkobkit (66070503489, GitHub: @Richyboy170 (https://github.com/Richyboy170))
- Tanakrit Sunthonwetchapong (67070503464, GitHub: @Tanakrit-triton (https://github.com/Tanakrit-triton))

Domain: QuickOrder System

1. Test Suite Overview & Verification Mapping

Test ID	Linked AC	Test Level	Preconditions / Input	Executed Action	Expected Result
TEST-AUTH-01	AC-AUTH-01	Integration / API	Seeded Customer account (customer@quickorder.local, Password123!).	POST /api/auth/login with correct email & password.	HTTP 200, valid session token returned, user.role == "CUSTOMER".
TEST-AUTH-02	AC-AUTH-02	Integration / API	None.	POST /api/auth/login with wrong password (WrongPass!).	HTTP 401 Unauthorized, error code INVALID_CREDENTIALS.
TEST-AUTH-03	AC-AUTH-03	Integration / API	Authenticated customer session token.	GET /api/auth/me with Bearer <token>.	HTTP 200, matching customer ID, email, and role.
TEST-AUTH-04	AC-AUTH-04	Integration / API	Authenticated session token.	POST /api/auth/logout, then subsequent GET /api/auth/me.	First call returns HTTP 200; subsequent call returns HTTP 401 Unauthorized.
TEST-MENU-01	AC-MENU-01	Integration / API	Database seeded with menu items.	GET /api/menu without auth header.	HTTP 200, array of items with id, name, price, stock ≥ 0.
TEST-ORD-01	AC-ORD-01	Integration / API	Customer token, item stock = 10.	POST /api/orders requesting quantity = 2.	HTTP 201 Created, status PENDING, server calculates correct total, stock reduced to 8.
TEST-ORD-02	AC-ORD-02	Unit / API	Customer token.	POST /api/orders with empty items: [] or quantity: 0.	HTTP 400 Bad Request, error INVALID_ITEMS (BR-01).
TEST-ORD-03	AC-ORD-03	Integration / API	Item has available stock = 1.	POST /api/orders requesting quantity = 5.	HTTP 400 Bad Request, error INSUFFICIENT_STOCK, stock remains 1 (BR-02).
TEST-ORD-04	AC-ORD-04	Integration / API	Item DB price = 149.00. Customer payload injects unitPrice: 1.00.	POST /api/orders with tampered pricing.	Stored order and response reflect $149.00 \times \text{qty}$, tampered price discarded (BR-03).
TEST-ORD-05	AC-ORD-05	Integration / API	Customer owns an order in PENDING state with item qty = 2.	POST /api/orders/:id/cancel by owner.	HTTP 200, order status becomes CANCELLED, item stock restored (+2).
TEST-ORD-06	AC-ORD-06	Integration / API	Order status is PREPARING (advanced by staff).	Customer attempts POST /api/orders/:id/cancel.	HTTP 400 Bad Request, error INVALID_TRANSITION, order stays PREPARING (BR-05).
TEST-SEC-01	AC-SEC-01	Integration / API	Customer A logged in. Order #200 belongs to Customer B.	Customer A calls GET /api/orders/200.	HTTP 403 Forbidden without exposing Customer B's order details (BR-04).
TEST-SEC-02	AC-SEC-02	Integration / API	Customer token.	Customer calls	HTTP 403 Forbidden,

				PATCH /api/orders/:id/status or GET /api/orders.	role guard rejects non-staff access (BR-06, NFR-02).
TEST-SEC-03	AC-SEC-03	Integration / API	No Authorization header.	Unauthenticated request to POST /api/orders.	HTTP 401 Unauthorized (NFR-01).
TEST-STAFF-01	AC-STAFF-01	Integration / API	Staff token (staff@quickorder.local).	GET /api/orders	HTTP 200, returns full list of orders across all customers.
TEST-STAFF-02	AC-STAFF-03	Integration / API	Staff token, order in PENDING.	PATCH /api/orders/:id/status to PREPARING.	HTTP 200, order status updated to PREPARING.
TEST-E2E-01	System Flow	E2E / UI	Clean seeded test environment.	1. Customer login; 2. Browse menu; 3. Add 2 burgers to cart; 4. Submit order; 5. Check status in My Orders.	Order appears in My Orders with PENDING status and correct price.
TEST-E2E-02	System Flow	E2E / UI	Staff user logged in.	1. Open Kitchen Queue; 2. View customer's order; 3. Advance to PREPARING; 4. Advance to READY; 5. Advance to COMPLETED.	Order cards update visually; customer sees updated status.

2. Test Execution Commands & Structure

- Backend Unit & Integration Tests:

```
cd server
npm test
```

Runs Vitest / Jest against in-memory or PostgreSQL test database:

- server/tests/auth.test.ts (Authentication, sessions, logout, 401/403)
- server/tests/order.test.ts (Order placement, stock reduction, price calculation, cancellation)
- server/tests/permissions.test.ts (Cross-customer boundary isolation, Staff queue access)
- Frontend Build & Lint Checks:

```
cd client
npm run build
```

Product Definition of Done

A Sprint 1 increment is done when required behavior is implemented, planned verification has run and passed, meaningful tests cover domain rules and permissions, peer review and staging integration are complete, setup is reproducible, and incomplete work or failed/skipped tests are disclosed.

DoD area	Required evidence
Specification and design	Versioned feature contracts, stable AC IDs, complete FR/BR/NFR traceability and justified changes.
Implementation	UI, application/API and durable database behavior; backend-enforced rules and role checks.
Verification	Build/test commands, tested commit, pass/fail/skip counts, AC-to-test map, UI/API/database evidence, two actual test excerpts and TDD red/green evidence where applicable.
Team workflow	Issues linked to contracts; feature branches from sprint staging; substantive teammate PR reviews; integration tests; accepted release PR to main; release commit.
Reproducibility	Current README, environment examples, migrations and seed instructions; synthetic test data; no secrets.

PART F — Working Sprint 1 and verification

Evidence cut-off: 14 November 2026. This section distinguishes the main-branch snapshot from the later persistence feature PR.

F.1 Release and repository state

Repository: https://github.com/Richyboy170/Software-Engineering-Project

Main and sprint-1-staging currently point to commit 1e0c693ad0161c4464a5ba6bc7e68720cfa6f584 (13 November 2026, 06:28 +07:00). No release tag is present. The Sprint 1 source and documents are in the repository, but the initialization commit contains the implementation and does not show separate contract/test-design and implementation steps or a reviewed Sprint 1 release.

In that main commit, the order route imports the in-memory store, which keeps users, sessions, menu items and orders in JavaScript Maps. Main also has a Prisma client and database readiness endpoint, but the domain routes do not use Prisma in this commit. Durable PostgreSQL persistence is on feature/prisma-persistence and is not integrated into main.

F.2 Current pull request and CI evidence

Reference	Observed state at evidence cut-off
Sprint 1 release branch	main and sprint-1-staging both point to 1e0c693ad0161c4464a5ba6bc7e68720cfa6f584; no Sprint 1 release PR or tag is recorded.
PR #5	Open: feature/prisma-persistence → sprint-1-staging; head 85845daea4a537032d468a2a9f9e93412895cce4. It is the Prisma persistence prerequisite, not a completed Sprint 1 release.
Teammate review	Tanakrit-triton is a requested reviewer on PR #5; GitHub returned no submitted reviews as of 14 November 2026.
CI	GitHub Actions test-and-docker completed successfully on PR #5 head on 13 November 2026. This checks the feature branch, not a released main commit.
Existing Sprint 1 PR table	The project evidence guide lists PRs #2, #3, #4 and #6 as Sprint 1 work. GitHub currently shows #2 and #3 as Dependabot PRs, #5 as the persistence PR, and no Sprint 1 PR #4 or #6. Those entries are not supported by current GitHub history.

Current pull request: https://github.com/Richyboy170/Software-Engineering-Project/pull/5

Successful CI run: https://github.com/Richyboy170/Software-Engineering-Project/actions/runs/43370877231/job/131706061952

F.3 Demonstration scenarios and evidence status

These scenarios are specified in the project evidence guide. Their expected results support a demonstration, but this report does not claim fresh screenshot-backed execution on the main release commit.

Scenario / IDs	Steps and expected result	Evidence status
Customer journey — F-01–F-04; AC-AUTH-01, AC-MENU-01, AC-ORD-01	Sign in as a seeded customer; browse menu; order two $149 burgers; retrieve the PENDING order. Expected total $298 and stock reduced by two.	Documented scenario; no current screenshot or main-release database trace captured.
Stock boundary — F-03; BR-02; TEST-ORD-03	Request five Truffle Parmesan Fries when seeded stock is two. Expected HTTP 400 INSUFFICIENT_STOCK; no order written and stock remains two.	Automated test exists; historical CI evidence is on PR #5 branch. No current local rerun.
Ownership/role boundary — F-04/F-06; BR-04/BR-06; TEST-SEC-01/02	Customer A requests Customer B's order and attempts a staff-only queue/status operation. Expected HTTP 403 without private order data.	Automated test source exists; historical CI evidence is on PR #5 branch. No current screenshot.
Logout — F-01; AC-AUTH-04; TEST-AUTH-04	Log in, call logout, then repeat a protected current-user request with the same token. Expected HTTP 200 for logout, then HTTP 401.	Automated test source exists; fresh screenshot not captured.

The rubric requires readable screenshots/screen dumps for the UI/API/database journey, a rule rejection, and login/logout plus a forbidden operation. Those screenshots are not included because the local Docker engine was unavailable during this report build.

F.4 Automated test and build evidence

Historical execution recorded in docs/sprints/sprint-2/persistence-evidence.md for feature/prisma-persistence on 13 November 2026:

```
npm --prefix server test
Test Files  5 passed (5)
     Tests  32 passed (32)

npm --prefix server run build
Completed successfully
```

The GitHub test-and-docker job for PR #5 also completed with conclusion success. These are historical feature-branch results; they do not establish that the main release commit passed the same tests.

Local attempts on 14 November 2026 could not start under the current environment: npm test stopped while esbuild tried to spawn a helper (spawn EPERM); npm run build completed the server TypeScript build, then client Vite/esbuild also stopped with spawn EPERM. Docker Compose could not connect to the Docker Desktop Linux engine. These are environment/tooling blocks, not reported as application test failures.

Test group / planned IDs	Source file(s)	Current evidence
Authentication and logout — TEST-AUTH-01–04	server/tests/auth.test.ts	Included in historical 32/32 PR #5 run; not rerun locally.
Menu, order, pricing, stock, cancellation — TEST-MENU-01; TEST-ORD-01–06	server/tests/order.test.ts	Included in historical 32/32 PR #5 run; test source is present.
Ownership, role and staff queue — TEST-SEC-01–03; TEST-STAFF-01–02	server/tests/permissions.test.ts	Included in historical 32/32 PR #5 run; test source is present.
PostgreSQL persistence — TEST-PERSIST-02–08	server/tests/prisma.integration.test.ts	Part of PR #5 feature branch; requires DATABASE_URL and PostgreSQL; not runnable locally because Docker is unavailable.

F.5 Selected actual test excerpts

Domain rule rejection — TEST-ORD-03 / AC-ORD-03, from server/tests/order.test.ts:

```
const res = await request(app)
  .post("/api/orders")
  .set("Authorization", `Bearer ${customerToken}`)
  .send({ items: [{ menuItemId: "menu_03", quantity: 5 }] });
expect(res.status).toBe(400);
expect(res.body.error.code).toBe("INSUFFICIENT_STOCK");
expect(store.getMenuItem("menu_03")!.stock).toBe(2);
```

Ownership restriction — TEST-SEC-01 / AC-SEC-01, from server/tests/permissions.test.ts:

```
const unauthorizedRes = await request(app)
  .get(`/api/orders/${orderId}`)
  .set("Authorization", `Bearer ${customer2Token}`);
expect(unauthorizedRes.status).toBe(403);
expect(unauthorizedRes.body.error.code).toBe("FORBIDDEN");
```

The CI summary records suite-level counts, not individual per-test output. These snippets are actual test source; this report does not claim they were re-executed locally on the main release commit.

F.6 Setup summary

The README documents Docker Compose startup, PostgreSQL 16, Prisma migration deployment, demo seeding, backend tests and TypeScript build. Example local verification commands:

```
docker compose up --build -d
npm test
npm run build
# API liveness: http://localhost:3000/api/health
# API readiness: http://localhost:3000/api/ready
# Client: http://localhost:5173 (or CLIENT_PORT=5174)
```

Real passwords, tokens and production secrets are not reproduced in this report. Use the documented local seed procedure for synthetic demo accounts.

F.7 Team contributions and AI-use declaration

Team member	Contribution recorded in project documents
Patiharn Liangkobkit (66070503489)	System-level SDS and architecture; authentication/authorization middleware; security and role tests; Docker/environment configuration; peer-review work.
Tanakrit Sunthonwetchapong (67070503464)	SRS and traceability; Prisma schema and seed; order and stock API; React customer/staff screens; peer-review work.

These responsibilities come from team project documents; current GitHub PR history does not independently corroborate the claimed Sprint 1 PR/review assignments.

AI-use declaration recorded in project documents: Claude / Gemini AI Coding Agents via Google Antigravity IDE were used to draft specification tables, generate TypeScript interfaces, structure relational models and design boundary tests. The team states both members reviewed generated material, validated rules/tests and conducted reciprocal code reviews.

F.8 Completion status and known gaps

Rubric evidence	Status at 14 Nov 2026
Parts A–E engineering documents	Present in repository and included in this PDF.
UI/API domain functionality	Source and automated tests are present; no fresh screenshot-backed execution is included.
Database persistence on main release	Not met: current main routes use in-memory state. Prisma persistence is only in open PR #5.
Build and test on submitted release commit	Not established. Historical 32/32 and CI pass apply to PR #5 head 85845da, not main commit 1e0c693.
UI/API/database, rejection and access screenshots	Missing from this report.
TDD red/green sequence for Sprint 1	Not demonstrated in available commit history.
Two completed Sprint 1 Issues and reviewed PRs	Not evidenced. Current PR history does not support the Sprint 1 entries in the existing evidence guide.
Reviewed staging integration and accepted release to main	Not completed in current GitHub state; main and staging both point to the initial Sprint 1 commit.
Known incomplete work disclosed	Yes; see this table and PR #5 status above.

This PDF is a self-contained evidence draft, not proof that every Deliverable 1 completion criterion is satisfied. The missing database release, real Sprint 1 review/integration history and screenshot evidence must be resolved or explicitly accepted by course staff before describing the deliverable as complete.